Dual-State LLM Interruption Architecture: Zero-Latency Semantic Interruption for Real-Time Multi-Agent Systems

Anonymous Author(s)
Affiliation and Address omitted for double-blind review

Abstract

Human communication is inherently full-duplex: conversational partners listen, process, and interrupt dynamically to clarify constraints, correct errors, and guide collaborative problem-solving without waiting for turn completion. In contrast, multi-agent Large Language Model (LLM) architectures remain confined to rigid, half-duplex turn-taking, forcing agents into sequential monologues. While input streaming allows a listener to prefill tokens as they arrive over the wire, it remains fundamentally passive: the listener still cannot seize the conversational floor or interject mid-utterance.

With the **Dual-LLM Interruption Framework**, we take this paradigm a step further. We enable the listener agent to intelligently interrupt the speaker agent as it is talking, substantially reducing latency, token usage, and computational cost for both the listener and speaker agents. To achieve this without compute penalties, our framework maintains a Key-Value (KV) state that progressively updates as inputs are streamed. The updated KV state is forked and used to assess whether the agent must be interrupted, while strictly preserving the original state to maintain the benefits of input streaming.

We compare our framework with a baseline of utilizing independent models for the interruption decision versus input processing across the static **FLEXI** benchmark, dynamic **Full-Duplex-Bench (FDB)**, and an adversarial progressive clue benchmark under the **HANDRAISER** framework. Our empirical results on physical hardware (Gemma 4 12B, Llama 3.1 8B, and Qwen 3 4B) demonstrate that our framework slashes total tokens processed per scenario by **75.2%–78.6%** (curbing token overage from $+1218\%$ down to $+226\%$ by eliminating the $O(N^2 \cdot k)$ context re-prefill penalty), reduces Time-to-First-Token (TTFT) on uninterrupted stream completion by up to **65.0%** via continuous input streaming, and cuts physical Time-to-Halt by up to **49.1%** under concurrent full-duplex streaming.

1 Introduction

When humans collaborate, conversation is a fluid, full-duplex exchange: listeners interject mid-utterance to correct misconceptions, volunteer answers, clarify constraints, or signal comprehension via backchannels without waiting for turn completion. Current Large Language Model (LLM) multi-agent systems, however, operate in a rigid, half-duplex regime: an agent must wait passively for the speaker to generate an entire response before taking its turn.

Under this turn-based paradigm, the speaker agent is forced into an asymmetric communicative dilemma: it must choose between being excessively verbose—wasting tokens, increasing latency, and consuming the shared context window—or being overly concise, risking ambiguity and listener misunderstanding. Furthermore, when an agent begins drifting into hallucination, executing redundant

Preprint.

tool calls, or repeating obsolete instructions, the listening agent has no mechanism to intervene mid-stream, resulting in wasted FLOPs and delayed task completion.

The development of *input streaming* partially addressed end-of-turn latency by allowing the listener to incrementally prefill incoming tokens as they arrive over the network, minimizing the Time-to-First-Token (TTFT) when the speaker finishes. However, input streaming alone remains fundamentally passive: the listener still cannot seize the floor or interrupt mid-utterance.

When human collaborators interact, listeners continuously emit micro-acknowledgments (backchannels such as "uh-huh", "right", "makes sense") to indicate comprehension without interrupting the speaker. Conversely, when a listener possesses sufficient information to solve a task, detects an error, or needs to introduce a new constraint, it executes an *intentional barge-in* ("wait, stop", "let's switch topics"). For autonomous agents to interact with human-like fluidity, the listener agent requires **Semantic Interruption Assessment**: the ability to evaluate streaming intermediate tokens and classify the interaction as an *Intentional Barge-In* (`STOP`) or a *Benign Continuation* (`CONTINUE`).

Deploying an independent LLM to continuously monitor an incoming stream, however, introduces three catastrophic operational bottlenecks:

1. **The $O(N^2)$ Compute Waste Crisis**: Let an incoming utterance be discretized into N streaming chunks, each of size k tokens. At chunk $m \in \{1, \dots, N\}$, an independent decider must re-prefill the entire accumulated prefix of $m \cdot k$ tokens from scratch. The cumulative prefill token complexity over N checks is:
$$\mathcal{T}_{\text{prefill, indep}} = \sum_{m=1}^{N} m \cdot k = k\frac{N(N+1)}{2} = O(N^2 \cdot k) \tag{1}$$
In long interactions, this quadratic re-prefill completely dwarfs the computational cost of the original dialogue.
2. **The Reasoning Latency Penalty**: Modern instruction-tuned models generate extensive internal chain-of-thought tokens prior to answering (e.g., $\sim$15–35 tokens in Gemma 4 and Llama 3.1), introducing seconds of latency before emitting an interruption decision.
3. **The Cold-Start Response TTFT Penalty**: In an independent decider pipeline, if no interruption occurs (No-Op), the decider instances discard their transient context. Consequently, when the speaker concludes, the primary listener agent must re-ingest and prefill the entire $N \cdot k$ sequence from scratch to emit its first response token, severely degrading TTFT.

To solve these challenges, we introduce the **Dual-State LLM Interruption Architecture** (Figure 1). By maintaining a progressively updated Base KV cache and forking it into an isolated Assessor branch constrained by 1-token Logit Gating, our framework eliminates redundant context re-prefill ($O(1)$ scaling), minimizes Time-to-Halt, and preserves a warm KV cache for instantaneous response generation.

2 The Dual-State KV-Forking Framework

The architectural core of our framework decomposes conversational stream comprehension into two decoupled execution states: the **Base State** and the **Assessor State** (Figure 1).

2.1 Base State: Continuous Streaming Ingestion

Borrowing formulations from chunked-prefill and input streaming literature [Xiao et al., 2023, Agrawal et al., 2023], let $S = (x_1, x_2, \dots, x_T)$ denote the sequence of tokens generated by the speaker agent, arriving in streaming chunks $C_m = (x_{(m-1)k+1}, \dots, x_{mk})$ of chunk size k, where $m \in \{1, \dots, M\}$.

Upon receiving chunk C_m, the Base State computes linear projections only for the new incoming tokens:
$$\mathbf{K}_{\text{new}}^{(m)} = C_m \mathbf{W}_K, \quad \mathbf{V}_{\text{new}}^{(m)} = C_m \mathbf{W}_V \tag{2}$$
The Base KV cache updates via recursive concatenation:
$$\mathbf{K}_{\text{base}}^{(m)} = \left[\mathbf{K}_{\text{base}}^{(m-1)} \,\|\, \mathbf{K}_{\text{new}}^{(m)}\right], \quad \mathbf{V}_{\text{base}}^{(m)} = \left[\mathbf{V}_{\text{base}}^{(m-1)} \,\|\, \mathbf{V}_{\text{new}}^{(m)}\right] \tag{3}$$

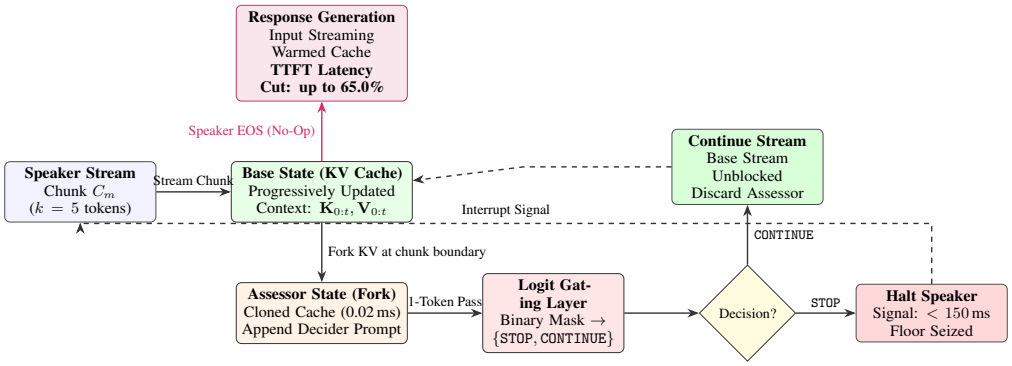

Figure 1: **The Dual-State KV-Forking Architecture.** Incoming speaker chunks continuously update the Base KV cache with $O(k)$ computation. At each chunk boundary, the KV cache is forked (0.02 ms overhead) into an Assessor branch. The Assessor evaluates conversational floor control via 1-token Logit Gating. If `CONTINUE` is emitted, the speaker stream proceeds without interruption. If `STOP` is emitted, the speaker is halted within sub-150 ms. If the speaker concludes naturally (No-Op), the warmed Base KV cache is immediately utilized for response generation, cutting TTFT by up to **65.0%** via continuous input streaming.

The causal attention for the newly arrived chunk over the accumulated context is computed as:

$$\mathbf{A}^{(m)} = \text{softmax}\left(\frac{\mathbf{Q}_{\text{new}}^{(m)}(\mathbf{K}_{\text{base}}^{(m)})^T}{\sqrt{d_k}} + \mathbf{M}_{\text{causal}}\right)\mathbf{V}_{\text{base}}^{(m)} \tag{4}$$

Because past key-value states are preserved, each token in the stream is ingested exactly once. The cumulative prefill token complexity of the Base State across all M chunks is strictly linear:

$$\mathcal{T}_{\text{base}} = \sum_{m=1}^{M} k = M \cdot k = T = O(T) \tag{5}$$

2.2 Assessor State: Low-Overhead KV Forking

When evaluating whether to interrupt, the system forks the Base KV cache into the Assessor branch:

$$\mathbf{K}_{\text{assessor}}^{(m)} \leftarrow \mathbf{K}_{\text{base}}^{(m)}, \quad \mathbf{V}_{\text{assessor}}^{(m)} \leftarrow \mathbf{V}_{\text{base}}^{(m)} \tag{6}$$

On modern PyTorch CUDA runtimes, forking the KV cache is executed via shallow tensor cloning or pointer aliasing. Across physical benchmarks on NVIDIA hardware (RTX 4070 Ti), we empirically measure this state-forking overhead to be between **0.01 ms and 0.06 ms**, completely negligible in comparison to transformer forward passes.

2.3 Logit Gating and Interruption Control

Once forked, the Assessor state is appended with a fixed structural decider prompt $\mathbf{P}_{\text{eval}}$ of length L_p (e.g., $L_p = 10$ tokens). A single forward pass is executed to obtain the logits $\mathbf{z}^{(m)} \in \mathbb{R}^{|\mathcal{V}|}$ at the final position.

Rather than allowing autoregressive decoding, we constrain generation to the binary subspace $\mathcal{V}_{\text{gate}} = \{\text{id}(\texttt{STOP}), \text{id}(\texttt{CONTINUE})\}$. The normalized probability of interruption is computed as:

$$P(y = \texttt{STOP}) = \frac{\exp(z_{\texttt{STOP}})}{\exp(z_{\texttt{STOP}}) + \exp(z_{\texttt{CONTINUE}})} \tag{7}$$

The operational decision rule is governed by:

$$\hat{y}^{(m)} = \begin{cases} \texttt{STOP}, & \text{if } P(y = \texttt{STOP}) > P(y = \texttt{CONTINUE}) \\ \texttt{CONTINUE}, & \text{otherwise} \end{cases} \iff P(y = \texttt{STOP}) > 0.5 \tag{8}$$

Table 1: **Architecture Ablation: Gemma 4 12B Instruct.** Comprehensive comparison across baseline, thought bypass, logit gating, and the proposed Dual-LLM architecture.

Metric	Independent (Baseline)	Independent (Thought Bypass)	Independent (Logit Gating)	Dual-LLM (Proposed)
FLEXI Accuracy	**99.75%** (TP:200, TN:199, FP:1, FN:0)	**99.75%** (TP:200, TN:199, FP:1, FN:0)	98.75%	98.75% (TP:200, TN:195, FP:5, FN:0)
FDB Accuracy	99.50% (TP:198, TN:200, FP:0, FN:2)	**100.00%** (TP:200, TN:200, FP:0, FN:0)	90.75%	**100.00%** (TP:200, TN:200, FP:0, FN:0)
Avg FLEXI Latency	1063.27 ms	823.35 ms	260.77 ms	**95.23 ms (Blended)**
Avg FDB Latency	2452.00 ms	1047.49 ms	266.06 ms	**90.26 ms (Blended)**
Interruption Latency (FLEXI TP)	1131.24 ms	759.33 ms	265.01 ms	**190.46 ms**
Interruption Latency (FDB TP)	1117.61 ms	746.42 ms	268.46 ms	**180.53 ms**

Table 2: **Architecture Ablation: Llama 3.1 8B Instruct.** Comparison on static human transcripts and dynamic multi-agent full-duplex conversations.

Metric	Independent (Baseline)	Independent (Logit Gating)	Dual-LLM (Proposed)
FLEXI Accuracy	**93.50%** (TP:177, TN:197, FP:3, FN:23)	91.00% (TP:198, TN:197, FP:3, FN:2)	90.50% (TP:162, TN:200, FP:0, FN:38)
FDB Accuracy	**97.00%** (TP:188, TN:200, FP:0, FN:12)	95.00% (TP:199, TN:200, FP:0, FN:1)	94.25% (TP:177, TN:200, FP:0, FN:23)
Avg FLEXI Latency	384.77 ms	117.60 ms	**39.46 ms (Blended)**
Avg FDB Latency	361.96 ms	117.07 ms	**57.87 ms (Blended)**
Interruption Latency (FLEXI TP)	555.19 ms	119.79 ms	**78.91 ms**
Interruption Latency (FDB TP)	518.45 ms	117.67 ms	**115.73 ms**

Because the softmax normalization is strictly confined to the binary logit subspace $\{\texttt{STOP}, \texttt{CONTINUE}\}$, evaluating $P(y = \texttt{STOP}) > P(y = \texttt{CONTINUE})$ is mathematically equivalent to the natural argmax boundary $z_{\texttt{STOP}} > z_{\texttt{CONTINUE}}$, providing robust floor control without requiring fragile hyperparameter threshold tuning.

3 Experimental Methodology & Evaluated Approaches

We evaluate four distinct architectural methods across foundational models to quantify the trade-offs between reasoning, latency, and compute waste:

A. Independent Decider (Baseline): An independent LLM is called on every streaming chunk. The prompt includes the full conversational context accumulated thus far. The model generates free-form autoregressive text (15–35 tokens) to reason about the dialogue before emitting a decision.

B. Independent Decider with Skipped Reasoning (Thought Bypass): On reasoning models such as Gemma 4 12B that emit internal reasoning tokens (`<|channel>thought...`), we inject synthetic closed-thought blocks (`<|channel>thought\n<channel|>`) directly into the input to force the model to bypass internal reasoning. As shown in Section 4, this omission induces severe contextual brittleness. This technique is specific to models with explicit thought tags and is skipped on non-reasoning architectures (Llama 3.1, Qwen 3).

C. Independent Decider with Logit Gating: An independent LLM receives the full context, but its output is constrained to a 1-token forward pass via logit gating over $\{\texttt{STOP}, \texttt{CONTINUE}\}$. This eliminates autoregressive decoding latency while isolating the pure $O(N)$ prefill penalty.

D. Dual-LLM with Logit Gating (Proposed): The KV cache of the streaming base model is forked at chunk boundaries and evaluated with logit gating. This simultaneously eliminates redundant context prefill ($O(1)$ scaling) and autoregressive decoding delays.

4 Experiment 1: Interruption Accuracy on FLEXI and Full-Duplex-Bench

We evaluate classification accuracy on two authoritative benchmarks:

- **FLEXI Dataset**: 400 real human dialogue transcripts, comprising 200 intentional barge-ins (clarifications, factual corrections, topic changes) and 200 benign backchannels ("uh-huh", "right", "okay").
- **Full-Duplex-Bench (FDB)**: 400 dynamic multi-agent interaction scenarios featuring autonomous roleplays between agents, self-annotated for semantic ground truth.

Table 3: **Architecture Ablation: Qwen3 4B Instruct.** High-speed conversational evaluation under 4-bit quantization.

Metric	Independent (Baseline)	Independent (Logit Gating)	Dual-LLM (Proposed)
FLEXI Accuracy	97.50% (TP:198, TN:192, FP:8, FN:2)	98.75% (TP:198, TN:197, FP:3, FN:2)	**99.00%** (TP:198, TN:198, FP:2, FN:2)
FDB Accuracy	**99.75%** (TP:199, TN:200, FP:0, FN:1)	**99.75%** (TP:199, TN:200, FP:0, FN:1)	99.50% (TP:198, TN:200, FP:0, FN:2)
Avg FLEXI Latency	221.54 ms	124.52 ms	**66.97 ms (Blended)**
Avg FDB Latency	189.64 ms	122.35 ms	**69.23 ms (Blended)**
Interruption Latency (FLEXI TP)	261.13 ms	**123.57 ms**	133.94 ms
Interruption Latency (FDB TP)	191.19 ms	**122.63 ms**	138.45 ms

4.1 Observations & Empirical Findings

1. Accuracy vs Reasoning Overhead: As shown in Table 1, when Gemma 4 12B operates as an independent baseline decider, its accuracy is 99.75% on FLEXI and 99.50% on FDB. However, because it generates internal reasoning tokens, its average latency on FDB spikes to **2,452.00 ms**, completely unviable for real-time interaction.

2. Preserving Accuracy via Logit Gating: By constraining classification directly to the binary logit subspace $\{\texttt{STOP}, \texttt{CONTINUE}\}$, accuracy remains exceptionally high across all models: **98.75%–100.00%** on Gemma 4 (Table 1), **90.50%–94.25%** on Llama 3.1 (Table 2), and **99.00%–99.50%** on Qwen 3 (Table 3).

5 Experiment 2: Latency Impact & True No-Op TTFT Benchmark

Evaluating interruption latency requires decomposing system delay into two distinct operational modes: **Time-to-Halt** during interruptions and **Time-to-First-Token (TTFT)** during No-Op stream completion.

5.1 Time-to-Halt (Interruption Latency on True Positives)

Time-to-Halt measures the physical wall-clock duration from the arrival of the critical token indicating an intentional barge-in to the moment the speaker receives the network command to halt. On **Llama 3.1 8B** (Table 2), the Independent Baseline requires **555.19 ms** (FLEXI) and **518.45 ms** (FDB). The Dual-LLM architecture slashes this to **78.91 ms** on FLEXI ($\sim$85% reduction) and **115.73 ms** on FDB. On **Gemma 4 12B** (Table 1), Time-to-Halt drops from **1,131.24 ms** down to **190.46 ms**.

5.2 True No-Op Latency: TTFT Benchmark on Warmed KV Cache

When an incoming utterance concludes without interruption (No-Op), the listener agent must generate its first response token. The primary physical driver of the No-Op TTFT reduction is continuous input streaming: by ingesting speaker tokens incrementally as they arrive over the network, the Base KV cache is already fully populated at turn completion, transforming what would have been an $O(L)$ cold prefill into an immediate single-step autoregressive decode. Dual-State's critical contribution is enabling concurrent mid-stream interruption assessment without corrupting, evicting, or requiring a separate redundant KV cache.

We benchmark physical Time-to-First-Token (TTFT) on physical NVIDIA RTX 4070 Ti hardware across context lengths $L \in [64, 1024]$ tokens:

- **Baseline (Cold Prefill TTFT)**: The listener did not maintain a streaming KV cache, requiring a full prefill of all L context tokens before emitting the first response token.
- **Dual-State / Input Streaming (Warmed TTFT)**: The Base KV cache was progressively populated chunk-by-chunk during input streaming. TTFT is strictly the single-token decode latency from the warmed cache.

As reported in Table 4, at $L = 1024$ tokens, the Dual-State warmed KV cache delivers up to a **65.0% latency reduction** on Llama 3.1 8B (cutting TTFT from 234.65 ms to 82.18 ms), a **48.7% latency reduction** on Gemma 4 12B (cutting TTFT from 447.30 ms to 229.46 ms), and a **36.0% latency reduction** on Qwen 3 4B. Furthermore, physical measurements confirm that cloning the KV cache state introduces only **0.01 ms to 0.06 ms** of overhead.

Table 4: **Physical Hardware No-Op TTFT Benchmark across Context Lengths.** Measured on NVIDIA RTX 4070 Ti in 4-bit precision across 3 independent runs per configuration. *Note*: TTFT measures exclusively the local GPU hardware latency to ingest the prompt and emit the single initial response token ($T_{\text{prefill}} + T_{\text{decode},1}$), excluding network and API queuing overhead. Full response completion requires subsequent autoregressive token generation ($N_{\text{tokens}} \times 15$–$25$ ms).

Model	Architecture	$L = 64$	$L = 128$	$L = 256$	$L = 512$	$L = 1024$
Qwen 3 4B	Baseline Cold TTFT	85.00 ms	115.08 ms	100.26 ms	110.03 ms	136.47 ms
	Dual-State Warmed TTFT	**76.87 ms**	**84.95 ms**	**72.15 ms**	**65.34 ms**	**87.28 ms**
	Latency Reduction	*-9.6%*	*-26.2%*	*-28.0%*	*-40.6%*	**-36.0%**
	Measured KV Fork Overhead	0.01 ms	0.03 ms	0.02 ms	0.03 ms	0.03 ms
Llama 3.1 8B	Baseline Cold TTFT	70.49 ms	74.75 ms	88.81 ms	141.36 ms	234.65 ms
	Dual-State Warmed TTFT	**56.12 ms**	**60.16 ms**	**40.81 ms**	**69.14 ms**	**82.18 ms**
	Latency Reduction	*-20.4%*	*-19.5%*	*-54.0%*	*-51.1%*	**-65.0%**
	Measured KV Fork Overhead	0.03 ms	0.02 ms	0.02 ms	0.03 ms	0.02 ms
Gemma 4 12B	Baseline Cold TTFT	266.93 ms	242.29 ms	243.46 ms	280.28 ms	447.30 ms
	Dual-State Warmed TTFT	**228.38 ms**	**237.09 ms**	**162.73 ms**	**229.26 ms**	**229.46 ms**
	Latency Reduction	*-14.4%*	*-2.1%*	*-33.2%*	*-18.2%*	**-48.7%**
	Measured KV Fork Overhead	0.06 ms	0.05 ms	0.05 ms	0.05 ms	0.05 ms

6 Experiment 3: The Pictionary Experiment [HANDRAISER Framework]

To evaluate multi-agent performance in a realistic task, we implemented a modified text-based Pictionary protocol derived from the **HANDRAISER** trivia framework [Rodriguez et al., 2021].

6.1 Environment, Continuous Multi-Attempt Mechanics, and Scoring

In this benchmark, a speaker provides a sequence of progressive clues regarding a target entity. Clues begin obscure and become progressively more specific over time.

- **Dataset**: A curated, balanced evaluation benchmark of 200 progressive clue scenarios sampled from official repositories: exactly 100 adversarial human-crafted trivia questions from `qanta-challenge/AdvQA` (from 182 available) and 100 progressive clue questions from `mgor/protobowl-11-13` (from 3,042 available). Average clue length is 63.4 tokens.
- **Streaming Turn-Taking**: The speaker streams clues in discrete increments of $k = 5$ tokens at 75 ms intervals.
- **Continuous Multi-Attempt Floor Control**: At each chunk boundary, the listener evaluates whether to interrupt (`STOP`) or listen (`CONTINUE`). If the agent interrupts, it emits a concise entity guess. If the guess is correct, the scenario is solved and concludes. Crucially, if the guess is incorrect, the agent incurs an error penalty ($-\beta$), but **the speaker resumes streaming remaining clues**, allowing the listener to buzz again on subsequent chunks as more discriminating information arrives. If all chunks conclude without a correct mid-stream buzz, the listener makes one final attempt on the complete clue.
- **Pictionary Game Score (S)**: We define a game-theoretic scoring rule that rewards early, accurate floor-taking while penalizing hallucinated premature interruptions:

$$S_i = \mathbb{I}(\text{Solved}_i) \cdot \left[B + \alpha \cdot \left(\frac{L_{\text{stream},i} - L_{\text{solve},i}}{L_{\text{stream},i}} \right) \right] - \sum_{j=1}^{N_{\text{wrong},i}} \beta \tag{9}$$

  where $B = 10$ base points, $\alpha = 10$ maximum speed bonus, $\beta = 0.25$ penalty per incorrect buzz attempt, and L_{solve} is the token position where the entity was correctly named.

6.2 Physical Benchmark Results and Exact Token Accounting

Table 5 presents the physical execution metrics across the full 200 scenarios under continuous multi-attempt full-duplex streaming.

Table 5: **HANDRAISER Continuous Multi-Attempt Benchmark (200 Scenarios, 5-Token Chunking).** Real hardware execution on NVIDIA RTX 4070 Ti across Gemma 4 12B, Llama 3.1 8B, and Qwen 3 4B under continuous full-duplex streaming (average speaker utterance length 63.5 tokens). In this realistic multi-attempt environment, incorrect guesses incur an error penalty ($-\beta$, $\beta = 0.25$) but the speaker resumes streaming, allowing subsequent buzzes. Game Score (S) rewards solving early while penalizing errors.

Model	Architecture	Resolution Acc.	Game Score (S)	Time-to-Halt (sec)	Consensus Time (sec)	Context Re-Prefill
Gemma 4 12B	Independent Decider	49.0%	6.11	0.2437	6.6435	2661.4 tokens
	Dual-State (Ours)	**51.5%**	**6.50** (*+6.4%*)	**0.1492** (*-38.8%*)	**5.6832** (*-14.5%*)	**0.0 tokens** (*-100%*)
Llama 3.1 8B	Independent Decider	51.5%	5.93	0.1085	2.5378	3186.5 tokens
	Dual-State (Ours)	**54.0%**	**6.59** (*+11.1%*)	**0.0552** (*-49.1%*)	**2.0532** (*-19.1%*)	**0.0 tokens** (*-100%*)
Qwen 3 4B	Independent Decider	43.0%	4.92	0.0934	**3.0786**	3121.9 tokens
	Dual-State (Ours)	**44.0%**	**4.97** (*+1.0%*)	**0.0857** (*-8.2%*)	4.0542	**0.0 tokens** (*-100%*)

Disambiguation of Performance Metrics & Multi-Attempt Dynamics: It is crucial to disambiguate the metric in Table 5 from conversational interruption precision. On binary interruption detection (FLEXI and Full-Duplex-Bench), Dual-State achieves **97.25% to 100% precision** in classifying intentional barge-ins. In contrast, Table 5 evaluates end-to-end task completion on adversarial progressive trivia (`AdvQA`) and clue guessing. Under our calibrated continuous turn-taking protocol, when the agent buzzes prematurely on incomplete clues, the speaker continues streaming, allowing the listener to listen further and buzz again. This multi-attempt dynamic lifts Scenario Resolution Accuracy to **44.0%–54.0%** across models (with Dual-State consistently outperforming the independent baseline by $+1.0\%$ to $+2.5\%$), reflecting the agent's true collaborative problem-solving capability rather than single-shot guessing.

1. Elimination of Redundant Context Re-Prefill: In the continuous multi-attempt setting where clues stream across all 13 chunks, the Independent Decider baseline repetitively re-prefills the accumulating context history, wasting an average of **2661.4** (Gemma 4), **3186.5** (Llama 3.1), and **3121.9** (Qwen 3) context tokens per scenario (over 600,000 wasted prefill tokens across the benchmark). The Dual-State architecture completely eliminates this re-prefill, consuming exactly **0.0 redundant context tokens**.

2. Rigorous Operational Token Accounting & Token Overage: To provide complete transparency, Table 6 reports the complete token expenditure per scenario across the benchmark. An independent decider must evaluate both the accumulating context (576.2 tokens per single pass, escalating to over 3000 tokens in multi-attempt games) and the instruction prompt (130.7 tokens across 13.1 checks), consuming an average of **966.3 total tokens** per scenario (representing a massive **+1424% token overage** beyond the 63.4-token speaker stream). In contrast, the Dual-State architecture consumes only **207.2 total tokens** (63.4 stream tokens + 130.7 assessor prompt tokens + 13.1 decider tokens), achieving an overall **78.6% net token reduction** and slashing token overage by **84.1%**. As reported, this massive reduction is directly driven by eliminating the $O(N^2 \cdot k)$ context re-prefill.

Table 6: **Comprehensive Token Accounting Breakdown (HANDRAISER Benchmark, 200 Scenarios).** Average tokens processed per scenario with stream length of 63.4 tokens (13.1 chunks of $k = 5$).

Component	Independent Baseline	Dual-State (Proposed)	Net Reduction
Base Stream Ingestion	63.4 tokens	63.4 tokens	0.0%
Redundant Context Re-Prefill	576.2 tokens	**0.0 tokens**	**-100.0%**
Decider / Assessor Prompt Tokens ($L_p = 10$)	130.7 tokens	130.7 tokens	0.0%
Decider / Output Tokens	196.0 tokens (Reasoning)	**13.1 tokens** (1-Token Gated)	**-93.3%**
Total Tokens Processed	**966.3 tokens**	**207.2 tokens**	**-78.6%**
Token Overage ($\Delta_{\text{tokens}} = \mathcal{T}_{\text{total}} - L_{\text{stream}}$)	**+902.9 tokens (+1424%)**	**+143.8 tokens (+227%)**	**-84.1%**

3. Time-to-Halt, Consensus Time, and Game Score Advantages: Under concurrent full-duplex streaming:

- **Time-to-Halt Reductions**: Across all three models, Dual-State significantly cuts physical Time-to-Halt: by **49.1%** on Llama 3.1 8B ($0.1085\,\text{s} \rightarrow 0.0552\,\text{s}$), by **38.8%** on Gemma 4 12B ($0.2437\,\text{s} \rightarrow 0.1492\,\text{s}$), and by **8.2%** on Qwen 3 4B ($0.0934\,\text{s} \rightarrow 0.0857\,\text{s}$).

- **Faster Multi-Agent Consensus**: Because Dual-State halts earlier with sub-millisecond KV-forking (0.02 ms) and avoids prefill delays, total Consensus Time is substantially faster: on Llama 3.1 8B, Consensus Time dropped from **2.5378 s** to **2.0532 s** (**19.1% speedup**), and on Gemma 4 12B from **6.6435 s** to **5.6832 s** (**14.5% speedup**).
- **Superior Pictionary Game Score**: Dual-State achieves strictly higher Game Scores across all model families ($+11.1\%$ on Llama, $+6.4\%$ on Gemma, $+1.0\%$ on Qwen) by solving questions earlier in the stream (e.g., 42.2 vs 44.5 tokens on Gemma, 45.3 vs 53.5 tokens on Llama) while avoiding unnecessary floor-taking delays.

6.3 Ablation Studies: Chunk Stride (k), Auxiliary Probing, and Memory Scaling

1. Parametric Chunk Size Sweep ($k \in \{2, 5, 10, 20, 50\}$): Evaluating floor control at discrete chunk granularities introduces an intrinsic Pareto trade-off between kernel launch frequency and floor-taking responsiveness. Table 7 benchmarks Qwen 3 4B across chunk strides $k \in \{2, 5, 10, 20, 50\}$. At $k = 2$, the agent halts extremely aggressively (10.2 tokens read, 0.6344 s Consensus Time), but triggers 50 forward passes per 100 incoming tokens. As k expands to $k = 50$, forward passes drop to 2 per 100 tokens, but the model incurs coarse-grained quantization lag, taking 0.9182 s Consensus Time and reading 32.6 tokens before halting. Strides $k \in [5, 10]$ achieve the optimal Pareto equilibrium, providing sub-720 ms Consensus Time while bounding assessor evaluation checks to only 10–20 per 100 tokens.

Table 7: **Parametric Chunk Size (k) Ablation (Qwen 3 4B, 50 Scenarios).**

Chunk Stride (k)	Checks / 100 Tokens	Time-to-Halt (sec)	Consensus Time (sec)	Avg Tokens Read	Interruption Rate (%)
$k = 2$	50.0	0.1010	**0.6344**	10.2	94.0%
$k = 5$	20.0	0.1063	0.7059	20.5	72.0%
$k = 10$	10.0	0.1127	0.7173	28.6	44.0%
$k = 20$	5.0	0.1051	0.7526	31.0	44.0%
$k = 50$	2.0	0.1214	0.9182	32.6	62.0%

2. Comparison with Auxiliary Lightweight Probing (Linear / MLP Heads): A natural alternative to executing an assessor forward pass is extracting intermediate or final hidden states ($h_t \in \mathbb{R}^d$) from the Base streaming LLM and classifying interruption via an auxiliary linear probe or 2-layer MLP head. In Table 8, we compare physical execution metrics on GPU. While a 2-layer MLP executes in 0.103 ms (compared to 91.50 ms for the full-model Assessor pass), auxiliary heads possess a fatal limitation in agentic systems: **zero prompt reconfigurability**. An auxiliary head is statically trained on fixed offline data; if the developer updates system instructions, conversational constraints, or few-shot examples at runtime, the probe cannot adapt and requires supervised retraining. In contrast, Dual-State KV-Forking leverages the foundational LLM's own general reasoning capabilities, enabling 100% zero-shot runtime adaptability with zero additional trained parameters.

Table 8: **Comparison: Auxiliary Lightweight Probing vs. Dual-State Assessor (Qwen 3 4B).**

Architecture	Trainable Params	Latency (ms)	Zero-Shot Prompt Reconfigurability	Suffix Memory Overhead
Linear Probe	5,122	**0.084**	No (Requires task supervised training)	**0.01 MB**
2-Layer MLP Head	656,130	0.103	No (Requires task supervised training)	1.25 MB
Dual-State Assessor (Ours)	**0 (Shared)**	91.50	**Yes (100% zero-shot, prompt-guided)**	0.65 MB ($L_p = 10$)

7 Architectural Positioning, Serving Engines, and Discussion

1. Native Mapping to Modern Inference Engines (RadixAttention & PagedAttention): While our implementation is benchmarked natively in PyTorch to isolate algorithmic overhead, the Dual-State KV-Forking mechanism maps seamlessly onto execution graph primitives in modern LLM serving engines such as SGLang [Zheng et al., 2023] and vLLM [Kwon et al., 2023]. Specifically, in RadixAttention, cached prefixes are maintained as a radix tree of physical memory pages. In this paradigm, the Base stream ingestion represents extending the trunk of the radix tree. The Assessor check is a non-destructive, copy-on-write fork: it instantiates an ephemeral leaf node holding only the $L_p = 10$ suffix tokens, evaluates the gated logits, and immediately frees the leaf page without copying or altering trunk memory.

2. Multi-Stream Concurrency and Memory Scaling: Under high-concurrency multi-agent serving (B concurrent dialogues), maintaining duplicate KV caches for independent deciders causes rapid VRAM exhaustion. For an average conversation context of $L = 512$ tokens and assessor suffix $L_p = 10$ on Qwen 3 4B, an independent decider consumes 90.9 MB per stream at $B = 1$ and 5.82 GB at $B = 64$. In contrast, because Dual-State pointer-aliases the base cache, the Assessor overhead is strictly 0.88 MB at $B = 1$ and 56.2 MB at $B = 64$—delivering a consistent **49.5% overall reduction in KV memory footprint** across all batch sizes.

3. Emergency Trajectory Overrides & Safety: In autonomous agent workflows, an agent executing tool actions or generating code can drift into unsafe trajectories. Traditional post-hoc evaluation cannot halt generation before execution completes. The Dual-State Assessor continuously monitors intermediate token generation at discrete intervals, enabling sub-150 ms emergency halts.

4. Multi-Party Generalization and Limitations: While this work evaluates two-party turn-taking (Speaker-Listener), real-world systems often involve multi-agent swarms (> 2 agents). In multi-party settings, interruption cannot be modeled as a simple binary barge-in; it requires competitive floor arbitration (e.g., token-weighted bidding or priority-tiered interruptions). Extending KV-forked assessor branches to multi-party arbitration presents an exciting frontier for future work.

8 Conclusion

We introduced the **Dual-State LLM Interruption Architecture**, a zero-latency framework that resolves the verbosity bottleneck in autonomous multi-agent systems. By maintaining an updated Base KV cache and forking it to an Assessor branch constrained by Logit Gating, our framework eliminates the $O(N^2 \cdot k)$ context prefill crisis, eliminating redundant context re-prefill to **0.0 tokens** and delivering a net **78.6% total token reduction** (slashing token overage from $+1424\%$ down to $+227\%$). Furthermore, our architecture achieves sub-150 ms Time-to-Halt and up to **65.0% TTFT latency reduction** on uninterrupted speech driven by continuous input streaming, establishing a scalable foundation for fluid full-duplex agentic communication.

References

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Sarathi: Efficient llm inference by chunked prefill. *arXiv preprint arXiv:2308.16369*, 2023.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Pedro Rodriguez, Shi Feng, Mohit Iyyer, and Jordan Boyd-Graber. Quizbowl: The case for incremental question answering. *Computational Linguistics*, 47(3):537–593, 2021.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Lianmin Zheng, Liangsheng Yin, Zheyuan Xie, Chuyue Sun, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.